

MetaMuseu

Galeria de Arte Virtual

Relatório Técnico — Projeto VR no Metaverso

Aluno:	Filipe Mazon
Turma:	Web 3.0 — Residência em TIC 29
Projeto:	Avançado — Unidade 1 Capítulo 3
Professora:	Ana Beatriz
Ano:	2026

Unity 6000.3.14f1

Meta XR SDK v201

XR Interaction Toolkit 3.0.7

URP 17

SEÇÃO 1 — IDENTIFICAÇÃO

Nome Completo:	Filipe Mazon
Turma / Residência:	Web 3.0 — Residência em TIC 29
Professora:	Ana Beatriz
Repositório GitHub:	https://github.com/f123r4/Meta_Galeriadearte_Filipe
Vídeo de Apresentação:	https://youtu.be/BPNqVuSoslI

Observação de Ambiente: Desenvolvimento realizado no Linux Bazzite com GPU Radeon RX 9060 XT. A limitação não é de hardware, mas de plataforma: o Meta XR Simulator é exclusivo para Windows e não está disponível no Linux. Por isso, a simulação de headset foi feita com o XR Device Simulator (XR Interaction Toolkit), que funciona no Linux e permite testar no Editor sem headset físico.

SEÇÃO 2 — CONCEITO DO PROJETO

2.1 Nome do Projeto

MetaMuseu — Galeria de Arte Virtual

2.2 Contexto e Objetivo no Metaverso

Criar um ambiente educacional e cultural imersivo no contexto do Metaverso, simulando uma galeria de arte virtual onde usuários podem visitar obras de arte digitais e explorar o espaço como se estivessem em um museu físico. O objetivo é demonstrar como o Metaverso pode democratizar o acesso à cultura e à arte, eliminando barreiras geográficas e financeiras.

2.3 Descrição Geral do Ambiente Virtual

Uma galeria de arte retangular com piso claro, teto branco e paredes bege. O espaço é dividido em ante-sala e salão principal. Na ante-sala, o visitante encontra o Validador de Ingresso — um totem com botão azul que libera o acesso ao salão. No salão principal, 6 obras de arte (quadros com moldura dourada, cada um com cor única) estão distribuídas nas paredes Norte, Leste e Oeste. A iluminação é quente (luz direcional com tom âmbar suave), com lightmaps baked pelo Progressive CPU Lightmapper.

SEÇÃO 3 — CONFIGURAÇÃO TÉCNICA DO PROJETO

3.1 Versão do Unity e Justificativa

Unity 6000.3.14f1 (Unity 6 LTS) — versão estável com suporte de longo prazo, recomendada para Meta XR SDK v201+ e com suporte nativo ao URP 17.

3.2 Instalação do Meta XR SDK

1. Window → Package Manager → "+" → Add package by name
2. Adicionado o escopo "com.meta.xr" no scopedRegistry do manifest.json apontando para <https://npm.developer.oculus.com>
3. Pacotes instalados: com.meta.xr.sdk.core (v201.0.0), com.meta.xr.sdk.interaction (v201.0.0), com.meta.xr.sdk.interaction.ovr (v201.0.0)
4. XR Interaction Toolkit (v3.0.7) instalado pelo Package Manager oficial do Unity

3.3 Configurações de Build para Android / Meta Quest

Configuração	Valor
--------------	-------

Platform	Android (Switch Platform aplicado)
Texture Compression	ASTC
Minimum API Level	Android 10 (API Level 29) — Quest 2/3
Target API Level	Android 12 (API Level 31)
Scripting Backend	IL2CPP
Target Architectures	ARM64 (único suportado pelo Quest)
Company Name	FilipeMazon
Product Name	MetaMuseu

3.4 Configuração do XR Plugin Management

- Edit → Project Settings → XR Plugin Management
- Aba PC (Windows): marcado OpenXR + Meta Quest Feature Group
- Aba Android: marcado OpenXR + Meta Quest Feature Group
- Interaction Profile adicionado: Oculus Touch Controller Profile

3.5 Movimentação no PC (Editor — Linux)

Utilizado CharacterController no PlayerController.cs — o objeto XROrigin (tag: Player) é movido por WASD/setas. Para simular XR no Editor (Linux): Window → Package Manager → XR Interaction Toolkit → Samples → XR Device Simulator → Import; prefab XR Device Simulator adicionado à cena. O Meta XR Simulator (ferramenta da Meta) é exclusivo para Windows e não está disponível no Linux Bazzite — por isso o XR Device Simulator do XR Interaction Toolkit foi utilizado como alternativa, sendo totalmente compatível com Linux.

SEÇÃO 4 — ASSETS E ELEMENTOS DA CENA

ASSET 1 — Quadros das Obras de Arte (6 unidades)

- Tipo:** Objeto 3D — Primitive Cube com material URP colorido
- Origem:** Primitive Unity com materiais gerados via MuseumBuilder.cs (Editor Script)
- Função:** Representar as 6 obras de arte expostas para contemplação no salão principal

ASSET 2 — Validador de Ingresso (totem de entrada)

- Tipo:** Objeto 3D composto — Cylinder + Cube + Sphere — com BotaoPrincipalController.cs
- Origem:** Primitive Unity com script de interação por proximidade
- Função:** Único ponto de interação — ao pressionar o botão, libera o acesso ao salão principal

ASSET 3 — HUD World Space (Canvas seguindo a câmera)

- Tipo:** UI / Canvas World Space com TextMeshPro
- Origem:** Unity UI com TextMeshPro e script HUDView.cs
- Função:** Exibir mensagens contextuais ao visitante durante a exploração do museu

SEÇÃO 5 — HIERARQUIA DE GAME OBJECTS

Cena: scene1.unity

```
[--- MANAGEMENT ---]
└─ GaleriaManager      → GaleriaManager.cs (Singleton central)
└─ EventSystem         → EventSystem + StandaloneInputModule
└─ HUD_Canvas          → Canvas (World Space) + HUDView.cs
    └─ Texto_Mensagem   → TextMeshProUGUI
```

```
[--- PLAYER ---]
└─ XROrigin          → CharacterController + PlayerController.cs | tag: Player
    └─ CameraOffset  → localPos (0, 1.7, 0)
        └─ MainCamera → Camera + AudioListener | tag: MainCamera

[--- ENVIRONMENT ---]
└─ Chao              → Plane (20×20 m)
└─ Teto              → Plane invertido (y = 5)
└─ Parede_Norte      → Cube (20.6 × 5 × 0.3)
└─ Parede_Leste/Oeste → Cube (0.3 × 5 × 20)
└─ Parede_Sul_Esq/Dir/Topo → 3 cubos formando parede com vão de entrada
└─ Directional_Light → Light (Directional, warm amber)

[--- EXIBITOS – estrutura de cada obra ---]
└─ Exibito_Harmonia_Cibernetica → ExibitoController + PainelInfoController
    │─ Moldura                  → Cube (borda dourada)
    │─ Tela                    → Cube + ExibitoView + XRSimpleInteractable
    │─ Gancho                  → Cylinder (suporte na parede)
    │─ ExhibitLight            → Light Spot
    └─ Painel_Info              → Canvas World Space + PainelInfoView
└─ Exibito_Floresta_Digital    → (mesma estrutura)
└─ Exibito_O_Sonho_do_Metaverso → (mesma estrutura)
└─ Exibito_Reflexos_do_Futuro   → (mesma estrutura)
└─ Exibito_Codigo_e_Alma        → (mesma estrutura)
└─ Exibito_Nexo_Quantico        → (mesma estrutura)

[--- INTERACTABLES ---]
└─ Validador_Ingresso → kiosque na ante-sala
    │─ Suporte_Coluna  → Cylinder
    │─ Suporte_Topo    → Cube
    │─ Botao_Ingresso  → Sphere + BotaoPrincipalController + BotaoPrincipalView
    └─ Label           → Canvas ("Validar Ingresso")
```

SEÇÃO 6 — SISTEMA DE INTERAÇÃO

6.1 Descrição da Interação Principal

A única interação disponível no MetaMuseu é o Validador de Ingresso na ante-sala. O visitante se aproxima do totem e pressiona o botão azul (Botao_Ingresso). O BotaoPrincipalController detecta o input, altera a cor do botão via BotaoPrincipalView e registra a ação no GaleriaManager. As 6 obras no salão principal são para contemplação — não possuem interação ativa.

6.2 Arquitetura MVC — Fluxo de Interação

```
Proximidade detectada por PlayerController (Physics.OverlapSphere)
↓
ExibitoController.AoEntrarHover()
↓
ExibitoController.Ativar() (apenas na primeira visita)
  +- view.AtivarDestaque() -----> ExibitoView (pulso de emissao verde)
  +- painelInfo.ExibirInfo() -----> PainelInfoController -> PainelInfoView (fade-in)
  +- GaleriaManager.Registrar()-> JogadorModel -> HUDView (mensagem + pontuacao)
```

6.3 Lógica de Interação — Passos

1. PlayerController move o XROrigin via WASD com CharacterController

2. Physics.OverlapSphere detecta objetos com tag interativa na zona do player
3. Ao detectar BotaoPrincipalController na zona, o botão muda para estado hover
4. Input.GetKeyDown(KeyCode.E) ou trigger XR → chama BotaoPrincipalController.Ativar()
5. BotaoPrincipalView altera a cor do botão para estado ativo
6. GaleriaManager.Instance?.ExibirMensagem() exibe confirmação no HUD

6.4 Script C# — BotaoPrincipalController.cs

```
using UnityEngine;

public class BotaoPrincipalController : MonoBehaviour
{
    private BotaoPrincipalView view;
    private bool ativado = false;

    private void Awake()
    {
        view = GetComponent<BotaoPrincipalView>();
    }

    public void Ativar()
    {
        if (ativado) return;
        ativado = true;
        view?.SetAtivo(true);
        GaleriaManager.Instance?.ExibirMensagem("Ingresso validado! Bem-vindo ao MetaMuseu.");
    }

    public void AoEntrarHover() => view?.SetHover(true);
    public void AoSairHover() => view?.SetHover(false);
}
```

SEÇÃO 7 — REPOSITÓRIO GITHUB

7.1 Dados do Repositório

Nome: Meta_Galeriadearte_Filipe

URL: https://github.com/f123r4/Meta_Galeriadearte_Filipe

7.2 Estrutura de Pastas

```
/Assets
  /Scripts
    /Models      → ExibitoModel.cs | JogadorModel.cs | AmbienteModel.cs
    /Views        → HUDView.cs | ExibitoView.cs | PainelInfoView.cs
                  ObjetoColetavelView.cs | BotaoPrincipalView.cs
    /Controllers  → ExibitoController.cs | PainelInfoController.cs
                  PortaAnimadaController.cs | ObjetoColetavelController.cs
                  BotaoPrincipalController.cs
  GaleriaManager.cs
  PlayerController.cs
  /Editor
    MuseumBuilder.cs      (menu MetaMuseu / Construir Cena Completa)
    LightingSetup.cs
    XRInteractableSetup.cs
```

```

/Scenes          → scene1.unity
/Materials/Museum → Materiais URP gerados pelo MuseumBuilder

/Packages/manifest.json → Meta XR SDK v201 + XR Interaction Toolkit 3.0.7
/ProjectSettings/      → apenas arquivos essenciais de build
.gitignore             → ignora /Library, /Temp, /Logs, /obj, /UserSettings

```

7.3 Arquitetura MVC — Tabela de Scripts

Script	Responsabilidade
GaleriaManager.cs	Singleton central — coordena estado global do museu
PlayerController.cs	Movimentação WASD + detecção por OverlapSphere
ExibitoModel.cs	Dados da obra (nome, descrição, pontos)
JogadorModel.cs	Pontuação, obras visitadas, tempo de sessão
AmbienteModel.cs	Estado global do ambiente (porta aberta/fechada)
HUDView.cs	HUD World Space — auto-limpa mensagens após 3s
ExibitoView.cs	Emissão URP + pulso verde ao ativar obra
PainelInfoView.cs	Canvas 3D com fade-in via CanvasGroup
BotaoPrincipalView.cs	Troca de cor: normal / hover / ativo
ExibitoController.cs	Interação com obras (XR + proximidade)
PainelInfoController.cs	Exibe painel, auto-fecha em 6s
PortaAnimadaController.cs	Animação Quaternion.Lerp, toggle
BotaoPrincipalController.cs	Lógica do validador de ingresso
MuseumBuilder.cs	Editor Script — constrói cena completa via menu

SEÇÃO 8 — PLANO DE EXECUÇÃO — PASSO A PASSO

1. Instalação do Unity 6000.3.14f1 e criação do projeto 3D com URP template.
2. Configuração do manifest.json com scoped registry da Meta e importação dos pacotes Meta XR SDK Core, Interaction e OVR via Package Manager.
3. XR Plugin Management — habilitar OpenXR nas abas PC e Android; adicionar Oculus Touch Controller Profile.
4. Build Settings → Switch Platform para Android; configurar Player Settings (API 29 min, ARM64, IL2CPP, ASTC).
5. Execução do menu MetaMuseu/► Construir Cena Completa para criar a hierarquia via MuseumBuilder.cs (Editor Script — único, auto-conecta tudo).
6. Materiais URP criados automaticamente pelo MuseumBuilder em Assets/Materials/Museum/; materiais únicos por obra preservam a cor no material instance criado em runtime.
7. Todas as referências (HUDView→GaleriaManager, textos→HUD, câmera→PlayerController) conectadas automaticamente por MuseumBuilder.ConectarReferencias().
8. Adicionar XRSimpleInteractable em cada exibito via menu MetaMuseu/► Configurar XRSimpleInteractable.
9. Testes no Editor com WASD (movimentação) e XR Device Simulator (simulação de headset).
10. Revisão final, push para GitHub e envio do link na plataforma Homero.

SEÇÃO 9 — REFLEXÃO FINAL

9.1 Aprendizado

Compreendi na prática como o XR Interaction Toolkit abstrai a complexidade do hardware VR, permitindo que os mesmos scripts funcionem no Editor (teclado) e no headset (controllers). A arquitetura MVC foi essencial para manter

os scripts organizados e reutilizáveis — separar `ExibitoModel` (dados), `ExibitoView` (visual) e `ExibitoController` (lógica) tornou muito mais fácil adicionar novas obras sem modificar código existente.

9.2 Dificuldades Encontradas

- Ausência de headset: o XR Device Simulator não representa fielmente as proporções do ambiente em VR — escala calibrada visualmente no Editor.
- Emissão URP em runtime: materiais precisam ter o keyword `_EMISSION` habilitado explicitamente via script; sem isso, `SetColor("_EmissionColor")` não tem efeito visual.
- Build Android no Linux (Bazzite): symlinks quebrados do clang++ e lld impediram o build e exigiram correção manual das toolchains NDK.
- Meta XR Simulator exclusivo para Windows: a limitação não foi de hardware (GPU Radeon RX 9060 XT no Linux Bazzite), mas de plataforma — o Meta XR Simulator não existe para Linux, sendo substituído pelo XR Device Simulator do XR Interaction Toolkit.
- Fade-in do painel: necessário usar `CanvasGroup.alpha` com `Mathf.MoveTowards`, pois `Color.Lerp` em `Image` não suporta transparência sem `CanvasGroup`.

9.3 Melhorias Futuras

- Multiplayer com Photon PUN2 para visitas simultâneas de vários usuários ao museu virtual.
- Áudio espacial (`AudioSource 3D`) com narração das obras via `AudioClip`.
- Sistema de teleporte entre salas para museus maiores.
- Integração de NFTs reais das obras via smart contract Ethereum — compra de obras diretamente no Metaverso.
- Suporte completo a controllers VR com raycast e grab para uso no Meta Quest físico.